Εθνικό Μετσόβιο Πολυτεχνείο
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Λειτουργικά Συστήματα Υπολογιστών

Γρίφος

Αλεξάνδρα Μυλωνά 03123004[*]
Ίωνας Τσακίρης 03123077[†]

13 Οκτωβρίου 2026

---

[*] el23004@mail.ntua.gr
[†] el23077@mail.ntua.gr

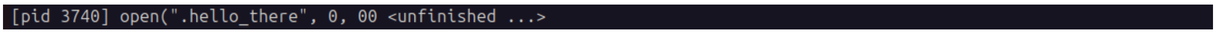
Σημείωση: Για τη διερεύνηση των εμποδίων χρησιμοποιούμε τα εργαλεία `ltrace` και `strace`, ώστε να παρακολουθούμε τις κλήσεις συναρτήσεων βιβλιοθηκών και τις κλήσεις συστήματος του εκτελέσιμου.

Η επίλυση αυτοματοποιείται μέσω προγράμματος σε C, το οποίο διαμορφώνει το κατάλληλο περιβάλλον εκτέλεσης και εκκινεί τον γρίφο σε διεργασία-παιδί με χρήση των fork() και execv().

Εμπόδιο 0

Διάγνωση

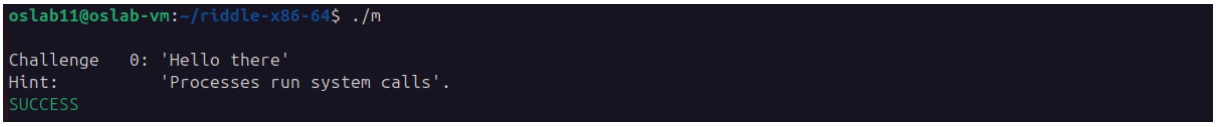
Χρησιμοποιώντας την `ltrace -SCf` βλέπουμε την κλήση `open(".hello_there", 0, 00) = -1`. Συμπεραίνουμε πως για την επίλυση απαιτείται η ύπαρξη του αρχείου `.hello_there` στον τρέχοντα κατάλογο εργασίας της διεργασίας.



```
[pid 3740] open(".hello_there", 0, 00 <unfinished ...>
```

Επίλυση

Δημιουργούμε το ζητούμενο αρχείο ώστε η κλήση να επιτυγχάνει, μέσω της `open()` με flag `O_CREAT`.



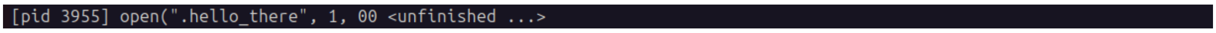
```
oslab11@oslab-vm:~/riddle-x86-64$ ./m

Challenge   0: 'Hello there'
Hint:          'Processes run system calls'.
SUCCESS
```

Εμπόδιο 1

Διάγνωση

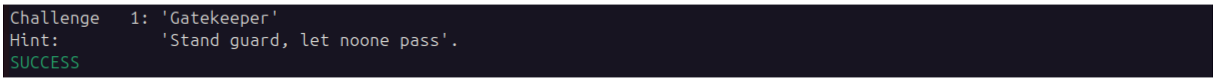
Όμοια, βλέπουμε πως γίνεται κλήση της `open(..., O_WRONLY, ...)` (δηλαδή ζητείται πρόσβαση εγγραφής στο αρχείο `.hello_there`.), την οποία θέλουμε να αποτρέψουμε.



```
[pid 3955] open(".hello_there", 1, 00 <unfinished ...>
```

Επίλυση

Αλλάζουμε την επίτρεψη του αρχείου σε 0444 (`r--r--r--`), δηλαδή σε read-only, χρησιμοποιώντας την `chmod()`.

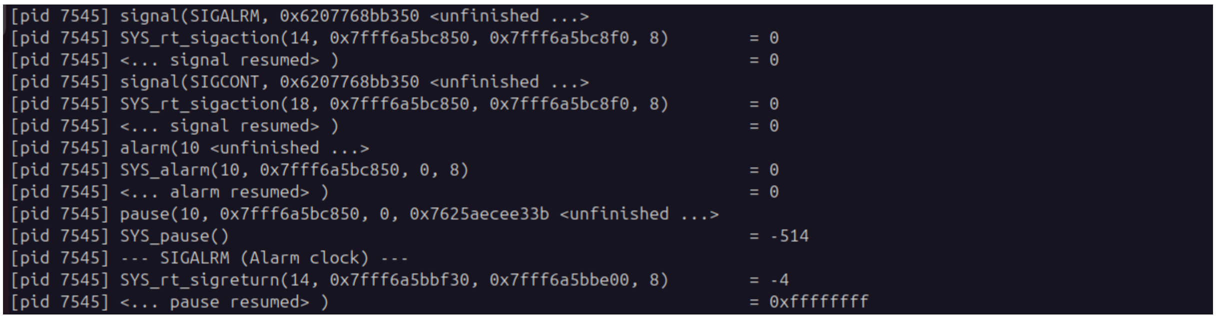


```
Challenge   1: 'Gatekeeper'
Hint:          'Stand guard, let noone pass'.
SUCCESS
```

Εμπόδιο 2

Διάγνωση

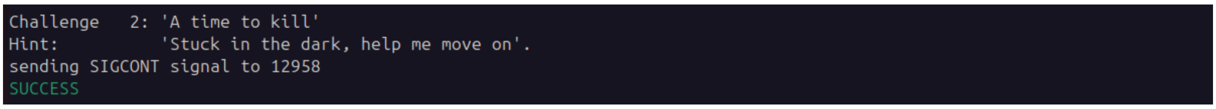
Από την κάτωθι έξοδο του `ltrace`, παρατηρούμε ότι ο γρίφος θέτει ένα `alarm(10)`, ένα `pause()` και handlers για τα `SIGCONT, SIGALRM`. Το `alarm(10)` προγραμματίζει την αποστολή του σήματος `SIGALRM` μετά από περίπου 10 δευτερόλεπτα, ενώ το `pause()` αναστέλλει την εκτέλεση της διεργασίας μέχρι να παραδοθεί κατάλληλο σήμα.



```
[pid 7545] signal(SIGALRM, 0x6207768bb350 <unfinished ...>
[pid 7545] SYS_rt_sigaction(14, 0x7fff6a5bc850, 0x7fff6a5bc8f0, 8)        = 0
[pid 7545] <... signal resumed> )                                         = 0
[pid 7545] signal(SIGCONT, 0x6207768bb350 <unfinished ...>
[pid 7545] SYS_rt_sigaction(18, 0x7fff6a5bc850, 0x7fff6a5bc8f0, 8)        = 0
[pid 7545] <... signal resumed> )                                         = 0
[pid 7545] alarm(10 <unfinished ...>
[pid 7545] SYS_alarm(10, 0x7fff6a5bc850, 0, 8)                            = 0
[pid 7545] <... alarm resumed> )                                          = 0
[pid 7545] pause(10, 0x7fff6a5bc850, 0, 0x7625aecee33b <unfinished ...>
[pid 7545] SYS_pause()                                                    = -514
[pid 7545] --- SIGALRM (Alarm clock) ---
[pid 7545] SYS_rt_sigreturn(14, 0x7fff6a5bbf30, 0x7fff6a5bbe00, 8)        = -4
[pid 7545] <... pause resumed> )                                          = 0xffffffff
```

Επίλυση

Εκτελούμε τον γρίφο ως παιδί ενός άλλου προγράμματος, το οποίο του στέλνει `SIGCONT` πρίν τη λήξη του χρονόμετρου, μέσω της `kill(p, SIGCONT)`. Όταν το σήμα παραδοθεί κατά την αναμονή στο `pause()`, ενεργοποιείται ο αντίστοιχος handler και η διεργασία μπορεί να συνεχίσει. Στην υλοποίησή μας χρησιμοποιείται η `sleep(1)` πριν από την αποστολή του σήματος, ώστε να δοθεί χρόνος στο παιδί να φτάσει στο `pause()`. Η καθυστέρηση αυτή αποδεικνύεται επαρκής στην πράξη, αλλά δεν εξασφαλίζει αυστηρό συγχρονισμό μεταξύ των διεργασιών.

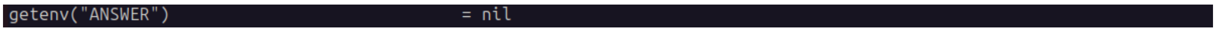


```
Challenge   2: 'A time to kill'
Hint:          'Stuck in the dark, help me move on'.
sending SIGCONT signal to 12958
SUCCESS
```

Εμπόδιο 3

Διάγνωση

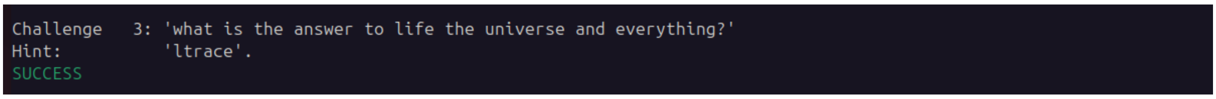
Όμοια με το εμπόδιο 6 του αντίστοιχου γρίφου του εισαγωγικού μαθήματος, παρατηρούμε ότι ο γρίφος καλεί την `getenv("ANSWER")`, η οποία επιστρέφει `NULL`, υποδεικνύοντας ότι η μεταβλητή περιβάλλοντος `ANSWER` δεν έχει οριστεί.



```
getenv("ANSWER")                             = nil
```

Επίλυση

Στο πρόγραμμα C χρησιμοποιούμε την `setenv("ANSWER", "42", 1)` πριν κάνουμε `fork()`. Το παιδί κληρονομεί το περιβάλλον του γονέα και το διατηρεί κατά την `execv()`, επιτρέποντας στον γρίφο να διαβάσει την επιθυμητή τιμή.

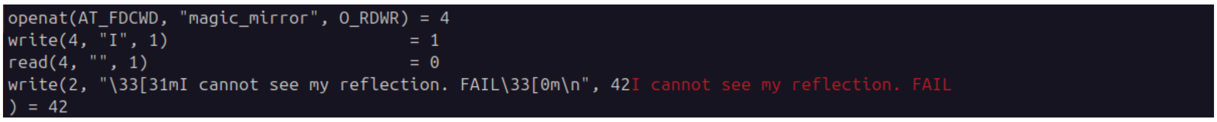


```
Challenge   3: 'what is the answer to life the universe and everything?'
Hint:          'ltrace'.
SUCCESS
```

Εμπόδιο 4

Διάγνωση

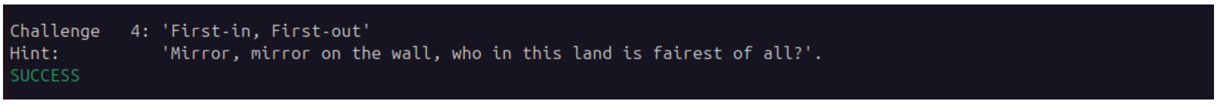
Ο γρίφος ανοίγει το αρχείο `magic_mirror` με `O_RDWR`, γράφει έναν χαρακτήρα και επιχειρεί να τον διαβάσει από τον ίδιο file descriptor. Σε ένα κανονικό αρχείο, η `write()` μετακινεί το file offset, με αποτέλεσμα η `read()` να επιστρέφει `0` (EOF).



```
openat(AT_FDCWD, "magic_mirror", O_RDWR) = 4
write(4, "I", 1)                        = 1
read(4, "", 1)                          = 0
write(2, "\33[31mI cannot see my reflection. FAIL\33[0m\n", 42I cannot see my reflection. FAIL
) = 42
```

Επίλυση

Δημιουργούμε ένα named FIFO pipe για τη διεργασία (βλ. `mkfifo(3)`) πριν από τη `fork()`. Σε αντίθεση με ένα κανονικό αρχείο, το FIFO μεταφέρει δεδομένα μέσω buffer του πυρήνα, χωρίς file offset, επιτρέποντας στον γρίφο να διαβάσει τον χαρακτήρα που μόλις έγραψε.



```
Challenge   4: 'First-in, First-out'
Hint:          'Mirror, mirror on the wall, who in this land is fairest of all?'.
SUCCESS
```

Εμπόδιο 5

Διάγνωση

Γίνεται κλίση της `fcntl` με `fildes = 99`.

Επίλυση

Δημιουργούμε ένα νέο αρχείο με δικαιώματα ανάγνωσης και γραφής, και με χρήση του `dup2()` του αναθέτουμε ως file descriptor τον αριθμό 99.

Εμπόδιο 6

Διάγνωση

Παρατηρούμε ότι ο γρίφος γεννά δύο διεργασίες, ο πρώτη εκ των οποίων τυπώνει το `<PID> PING!` στο `stdout` και γράφει στο file descriptor 34, ενώ η δεύτερη θέτει ένα χρονόμετρο των

2 δευτερολέπτων πριν διαβάσει από το file descriptor 33. Η διαδικασία αυτή επαναλαμβάνετε με ολοένα αυξανόμενες τιμές file descriptors.

```
[pid 23930] __fprintf_chk(0x7f3b9081a4a0, 1, ”Hint:----------’%s’.\n”, ”help-us-play!”Hint:
    ’help-us-play!’.
) = 32
[pid 23930] getenv(”I_NEED_TECH_HINTS”) = nil
[pid 23930] fork() = 23931
[pid 23930] fork( <unfinished ...>
[pid 23931] <... fork resumed> ) = 0
[pid 23931] getpid( <unfinished ...>
[pid 23930] <... fork resumed> ) = 23932
[pid 23930] wait(0x7ffcb7efaa74 <unfinished ...>
[pid 23932] <... fork resumed> ) = 0
[pid 23932] alarm(2 <unfinished ...>
[pid 23931] <... getpid resumed> ) = 23931
[pid 23931] __printf_chk(1, ”[%d]-PING!\n”, 23931 <unfinished ...>
[pid 23932] <... alarm resumed> ) = 0
[pid 23932] read(33 <no return ...>
error: maximum array length seems negative
, ”admi”, 4) = -1
[pid 23932] alarm(0) = 2
[23931] PING!
[pid 23932] exit(1 <unfinished ...>
[pid 23931] <... __printf_chk resumed> ) = 14
[pid 23931] write(34, ””, 4 <no return ...>
[pid 23932] +++ exited (status 1) +++
[pid 23930] --- SIGCHLD (Child exited) ---
[pid 23930] <... wait resumed> ) = 23932
[pid 23930] wait(0x7ffcb7efaa74 <unfinished ...>
[pid 23931] <... write resumed> ) = -1
[pid 23931] exit(1 <no return ...>
[pid 23931] +++ exited (status 1) +++
[pid 23930] --- SIGCHLD (Child exited) ---
[pid 23930] <... wait resumed> ) = 23931
```

Επίλυση

Δημιουργούμε μια συστοιχία pipes με κατάλληλες τιμές file descriptor, τα οποία κληρονομούν αφ΄ ενός ο γρίφος, αφ΄ ετέρου τα παιδιά του.

Εμπόδιο 7

Διάγνωση

```
[pid 6489] __lxstat(1, ”.hello_there”, 0x7ffedf210b40 <unfinished ...>
[pid 6489] lstat@SYS(”.hello_there”, 0x7ffedf210b40) = 0
[pid 6489] <... __lxstat resumed> ) = 0
[pid 6489] __lxstat(1, ”.hey_there”, 0x7ffedf210bd0 <unfinished ...>
[pid 6489] lstat@SYS(”.hey_there”, 0x7ffedf210bd0) = 0
[pid 6489] <... __lxstat resumed> ) = 0
```

Επίλυση

Δημιουργούμε το hardlink: .hey_there → .hello_there.

Εμπόδιο 8

Διάγνωση

```
[pid 8452] open(”bf00”, 0, 021 <unfinished ...>
[pid 8452] openat@SYS(AT_FDCWD, ”bf00”, 0, 00) = 4
[pid 8452] <... open resumed> ) = 4
[pid 8452] lseek(4, 1073741824, SEEK_SET <unfinished ...>
[pid 8452] lseek@SYS(4, 1073741824, 0) = 1073741824
[pid 8452] <... lseek resumed> ) = 1073741824
[pid 8452] read(4 <unfinished ...>
[pid 8452] read@SYS(4, ””, 16) = 0
[pid 8452] <... read resumed> , ””, 16) = 0
[pid 8452] __printf_chk(1, ”%s”, ”X” <unfinished ...>
[pid 8452] fstat@SYS(1, 0x7fff263104a0) = 0
[pid 8452] <... __printf_chk resumed> ) = 1
[pid 8452] fflush(0x7f8a7961a580 <unfinished ...>
[pid 8452] write@SYS(1, ”X”, 1X) = 1
[pid 8452] <... fflush resumed> ) = 0
[pid 8452] close(4 <unfinished ...>
[pid 8452] close@SYS(4) = 0
[pid 8452] <... close resumed> ) = 0
...
[pid 8452] write@SYS(1, ”X”, 1X) = 1
[pid 8452] <... fflush resumed> ) = 0
[pid 8452] close(4 <unfinished ...>
[pid 8452] close@SYS(4) = 0
[pid 8452] <... close resumed> ) = 0
[pid 8452] open(”bf00”, 0, 00 <unfinished ...>
[pid 8452] openat@SYS(AT_FDCWD, ”bf00”, 0, 00) = 4
[pid 8452] <... open resumed> ) = 4
[pid 8452] lseek(4, 1073741824, SEEK_SET <unfinished ...>
[pid 8452] lseek@SYS(4, 1073741824, 0) = 1073741824
[pid 8452] <... lseek resumed> ) = 1073741824
[pid 8452] read(4 <unfinished ...>
[pid 8452] read@SYS(4, ””, 16) = 0
```

```
[pid 8452] <... read resumed> , ””, 16) = 0
[pid 8452] __printf_chk(1, ”%s”, ”X”) = 1
[pid 8452] fflush(0x7f8a7961a580 <unfinished ...>
[pid 8452] write@SYS(1, ”X”, 1X) = 1
[pid 8452] <... fflush resumed> ) = 0
[pid 8452] close(4 <unfinished ...>
[pid 8452] close@SYS(4) = 0
[pid 8452] <... close resumed> ) = 0
```

Επίλυση

Πρέπει να φτιάξουμε ένα (σύστημα) αρχείο(/ων) που υποστηρίζει εγγραφή/ανάγνωση σε offset 1073741824 (0x40000000). Στην προκειμένη περίπτωση, αξιοποιούμε τη δυνατότητα του ext4 να δημιουργεί αραιά αρχεία χωρίς να τα γεμίζει με μηδενικά. Εναλλακτικά, θα μπορούσαμε να φτιάξουμε ένα σύστημα ψεύδο-αρχείων με χρήση του FUSE, τα οποία φαινομενικά έχουν μέγεθος 1GB.

Εμπόδιο 9

Διάγνωση

Έξοδος `ltrace -SCf`:

```
[pid 14986] socket(2, 1, 0 <unfinished ...>
[pid 14986] socket@SYS(2, 1, 0, 0) = 4
[pid 14986] <... socket resumed> ) = 4
[pid 14986] inet_aton(”127.0.0.1”, {0x100007f}) = 1
[pid 14986] fputs(”I-am-trying-to-contact-you...\\n”, 0x7fb1ae41a4a0 <unfinished ...>
[pid 14986] write@SYS(2, ”I-am-trying-to-contact-you...\\n”, 31I am trying to contact
    you...\n) = 31
[pid 14986] <... fputs resumed> ) = 1
[pid 14986] connect(4, 0x7ffe7e4c6250, 16, 0x7fb1ae41b740 <unfinished ...>
[pid 14986] connect@SYS(4, 0x7ffe7e4c6250, 16, 0) = −111
[pid 14986] <... connect resumed> ) = 0xffffffff
[pid 14986] fputs(”...but-you-don’t-seem-to-listen.”..., 0x7fb1ae41a4a0 <unfinished ...>
[pid 14986] write@SYS(2, ”...but-you-don’t-seem-to-listen.”..., 34...but you don’t-seem-
    to-listen.\n)-----------------------------------=-34
[pid-14986]-<...-fputs-resumed>-)-----------------------------------------------------------------=-1
```

Έξοδος `strace -f -e trace=network -s 128`:

```
socket(AF_INET, SOCK_STREAM, IPPROTO_IP) = 3
bind(3, {sa_family=AF_INET, sin_port=htons(0), sin_addr=inet_addr(”127.0.0.1”)},
    16) = 0
listen(3, 8) = 0
strace: Process 13584 attached
[pid 13583] accept(3
...
```

```
Challenge 9: ’Connect’
Hint: ’Let-me-whisper-in-your-ear’.
 <unfinished ...>
[pid 13584] socket(AF_INET, SOCK_STREAM, IPPROTO_IP) = 4
I am trying to contact you...\n[pid 13584] connect(4, {sa_family=AF_INET,
    sin_port=htons(49842), sin_addr=inet_addr(”127.0.0.1”)}, 16) = −1
    ECONNREFUSED (Connection refused)
...but you don’t-seem-to-listen.\nFAIL
```

Επίλυση

Δημιουργούμε ένα TCP server στο 127.0.0.1:49842, βάσει της εξόδου του `strace`, οπότε δεχόμαστε ένα μήνυμα της μορφής ”How much is X + Y?”, και απαντάμε με το άθροισμα των δύο αριθμών.

Εμπόδιο 10

Διάγνωση

```
openat(AT_FDCWD, ”secret_number”, O_RDWR|O_CREAT|O_TRUNC, 0600) = 4
unlink(”secret_number”) = 0
write(4, ”The-number-I-am-thinking-of-right-now-is-[uppercase-matters]:-
    \0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\
    4096) = 4096
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_SHARED, 4, 0) =
    0x7f46b8703000
close(4) = 0
write(2, ”What-hex-number-am-I-thinking-of-right-now?-”, 44What hex number am I
    thinking of right now? ) = 44
fstat(0, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}) = 0
read(0BE3F5154
, ”BE3F5154\n”, 1024) = 9
write(2, ”\33[32mSUCCESS\33[0m\n”, 17SUCCESS
```

Επίλυση

Παρατηρούμε ότι ο γρίφος δημιουργεί ένα αρχείο `secret_number`, το οποίο στη συνέχεια σβήνει και μετά γράφει τον μυστικό αριθμό. Δημιουργούμε ένα hard link προς το `secret_number` και λαμβάνουμε τον αριθμό.

Εναλλακτικά, παρατηρούμε ότι χαρτογραφεί το αρχείο στη μνήμη του, οπότε μπορούμε να το αναζητήσουμε στο `/proc/<riddle_pid>/`.

Εμπόδιο 11

Διάγνωση

Όμοια με το Εμπόδιο 10, μόνο που γίνεται έλεγχος του reference count του αρχείου, οπότε δεν μπορούμε να χρησιμοποιήσουμε την άνω μέθοδο.

Επίλυση

Αναζητούμε τον αριθμό στη χαρτογραφημένη μνήμη της διεργασίας. Αλλά το κάνουμε μέσω της bash (βλ. αρχείο `C11.sh`), και όχι διαμέσου της διαμοιραζόμενης μνήμης, γιατί η πρώτη μέθοδος είναι πιο ενδιαφέρουσα :Δ.

Εμπόδιο 1

Διάγνωση

Επίλυση